Lecture 45-46:
Solving Linear Nonhomogeneous and DAC Recurrence Relations

Zhilin Wu (吴志林),
Key Laboratory of System Software,
Institute of Software,
Chinese Academy of Sciences

Outline

- Solving linear nonhomogeneous recurrence relations
- Recurrence relations resulting from Divide-and-Conquer (DAC) algorithms
- Solving DAC recurrence relations
- Closest-Pair Problem

Linear Homogeneous Recurrence Relations Associated with Linear Nonhomogeneous Ones

Let $a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k} + f(n)$
with $c_1, \ldots, c_k \in \mathbb{R}$
be a linear nonhomogeneous recurrence relation.
Its *associated homogeneous linear recurrence relation* is

$$a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k}.$$

The associated homogeneous recurrence relation of $H_n = 2H_{n-1} + 1$ is

$$H_n = 2H_{n-1}$$

Solving Linear Nonhomogeneous Recurrence Relations

Example. Solving $H_n = 2H_{n-1} + 1$.

Solving Linear Nonhomogeneous Recurrence Relations

Example. Solving $H_n = 2H_{n-1} + 1$.

$\{2^n - 1\}$ is a solution of $H_n = 2H_{n-1} + 1$.

Solving Linear Nonhomogeneous Recurrence Relations

Example. Solving $H_n = 2H_{n-1} + 1$.

$\{2^n - 1\}$ is a solution of $H_n = 2H_{n-1} + 1$.

The associated homogeneous linear recurrence relation is

$$H_n = 2H_{n-1}.$$

The solutions of $H_n = 2H_{n-1}$ are of the form $\{c2^n\}$ with $c \in \mathbb{C}$.

Solving Linear Nonhomogeneous Recurrence Relations

Example. Solving $H_n = 2H_{n-1} + 1$.

$\{2^n - 1\}$ is a solution of $H_n = 2H_{n-1} + 1$.

The associated homogeneous linear recurrence relation is

$$H_n = 2H_{n-1}.$$

The solutions of $H_n = 2H_{n-1}$ are of the form $\{c2^n\}$ with $c \in \mathbb{C}$.

For every $c \in \mathbb{C}$, $\{(2^n - 1) + c2^n\}$ is a solution of $H_n = 2H_{n-1} + 1$.

$H_n = (2^n - 1) + c2^n$, $2H_{n-1} + 1 = 2((2^{n-1} - 1) + c2^{n-1}) + 1 = (2^n - 1) + c2^n$

Solving Linear Nonhomogeneous Recurrence Relations

Example. Solving $H_n = 2H_{n-1} + 1$.

$\{2^n - 1\}$ is a solution of $H_n = 2H_{n-1} + 1$.

The associated homogeneous linear recurrence relation is

$$H_n = 2H_{n-1}.$$

The solutions of $H_n = 2H_{n-1}$ are of the form $\{c2^n\}$ with $c \in \mathbb{C}$.

For every $c \in \mathbb{C}$, $\{(2^n - 1) + c2^n\}$ is a solution of $H_n = 2H_{n-1} + 1$.

$H_n = (2^n - 1) + c2^n$, $2H_{n-1} + 1 = 2((2^{n-1} - 1) + c2^{n-1}) + 1 = (2^n - 1) + c2^n$

In particular, $\{-1\}$ is a solution of $H_n = 2H_{n-1} + 1$.

Solving Linear Nonhomogeneous Recurrence Relations

Theorem.

Let $a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k} + f(n)$

with $c_1, \ldots, c_k \in \mathbb{R}$

be a linear nonhomogeneous recurrence relation and $\{h_n^{(p)}\}$ be one of its solutions. Then every solution of

$$a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k} + f(n)$$

is of the form $\{h_n^{(p)} + h_n^{(q)}\}$, where $\{h_n^{(q)}\}$ is a solution of its associated homogeneous recurrence relation.

Solving Linear Nonhomogeneous Recurrence Relations

Proof.

Suppose $\{h_n^{(p)}\}$ be a particular solution of

$$a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k} + f(n).$$

Then $h_n^{(p)} = c_1 h_{n-1}^{(p)} + \cdots + c_k h_{n-k}^{(p)} + f(n)$.

Let $\{g_n\}$ be a solution of this recurrence relation. Then

$$g_n = c_1\, g_{n-1} + c_2\, g_{n-2} + \ldots + c_k\, g_{n-k} + f(n).$$

Therefore, we have

$$g_n - h_n^{(p)} = c_1 \left(g_{n-1} - h_{n-1}^{(p)} \right) + \cdots + c_k \left(g_{n-1} - h_{n-k}^{(p)} \right).$$

Let $h_n^{(q)} = g_n - h_n^{(p)}$. Then $g_n = h_n^{(p)} + h_n^{(q)}$ and $h_n^{(q)}$ is a solution of $a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k}$.

Solving Linear Nonhomogeneous Recurrence Relations

How to find a solution of

$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k} + f(n)$?

No general method for finding such a solution that works for every $f(n)$

There are techniques that work for certain types of $f(n)$

e.g. when $f(n)$ is the product of a polynomial of n and the nth power of a constant.

Solving Linear Nonhomogeneous Recurrence Relations

Theorem.

Let $a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k} + f(n)$ with $c_1, \ldots, c_k \in \mathbb{R}$ be a linear nonhomogeneous recurrence relation.

Moreover, suppose $f(n) = (b_t n^t + b_{t-1} n^{t-1} + \cdots + b_1 n + b_0) s^n$ with $b_t, \ldots, b_0, s \in \mathbb{R}$.

If s is not the root of the characteristic equation of

$$a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k},$$

then there is a solution of the nonhomogeneous recurrence relation of the form $\left\{(p_t n^t + p_{t-1} n^{t-1} + \cdots + p_1 n + p_0) s^n\right\}$,

otherwise, let m be the multiplicity of s, then there is a solution of the form $\left\{n^m (p_t n^t + p_{t-1} n^{t-1} + \cdots + p_1 n + p_0) s^n\right\}$,

where $p_t, \ldots, p_0 \in \mathbb{C}$.

Solving Linear Nonhomogeneous Recurrence Relations

For a proof, please read Chapter 3 of the following book

Paul Cull, Mary Flahive, Robby Robson,
Difference equations: From Rabbits to Chaos,
Undergraduate Texts in Mathematics,
Springer, 2004.

Solving Linear Nonhomogeneous Recurrence Relations

Example. Find a solution of $a_n = 6a_{n-1} - 9a_{n-2} + n2^n$

Solving Linear Nonhomogeneous Recurrence Relations

Example. Find a solution of $a_n = 6a_{n-1} - 9a_{n-2} + n3^n$

Solving Linear Nonhomogeneous Recurrence Relations

Care must be taken when $s = 1$, i.e.

$$f(n) = (b_t n^t + b_{t-1} n^{t-1} + \cdots + b_1 n + b_0)\ 1^n$$
$$= b_t n^t + b_{t-1} n^{t-1} + \cdots + b_1 n + b_0$$

Then the form of the solution depends on whether 1 is the root of the characteristic equation of the associated homogeneous recurrence relation.

Solving Linear Nonhomogeneous Recurrence Relations

Example. Consider $a_n = \sum_{k=1}^{n} k^2$. Then $a_n = a_{n-1} + n^2$.

Outline

- Solving linear nonhomogeneous recurrence relations
- **Recurrence relations resulting from Divide-and-Conquer (DAC) algorithms**
- Solving DAC recurrence relations
- Closest-Pair Problem

Binary search

procedure *bsearch*(x, a_1, …, a_n)
{x is an integer, a_1, …, a_n are increasing integers}
if $n = 1$ **then**
  if $x = a_1$ **then return** 1 **else return** 0
else
$m := \lfloor n/2 \rfloor$
if $x > a_m$ **then**
  temp := *bsearch*(x, a_{m+1}, …, a_n)
  if *temp* > 0 **then**
    location := m+*temp*
  else *location* := 0
else
  location := *bsearch*(x, a_1, …, a_m)
return *location*

Binary search

procedure *bsearch*(x, a_1, …, a_n)
{x is an integer, a_1, …, a_n are increasing integers}
if $n = 1$ **then**
  if $x = a_1$ **then return** 1 **else return** 0
else
  $m := \lfloor n/2 \rfloor$
  if $x > a_m$ **then**
    temp := *bsearch*(x, a_{m+1}, …, a_n)
    if *temp* > 0 **then**
      location := m+*temp*
    else *location* := 0
  else
    location := *bsearch*(x, a_1, …, a_m)
  return *location*

$f(n)$: the number of comparisons of *bsearch*(x, a_1, …, a_n)

$$f(n) = f(n/2) + 3$$

if n is even

Finding the Maximum and Minimum

procedure *Maxmin*(a_1, …, a_n: real numbers with $n \geq 1$)
if $n = 1$ **then**
  $x := a_1$ $y := a_1$
else
  $m := \lfloor n/2 \rfloor$
  $(x_1, y_1) := Maxmin(a_1, \ldots, a_m)$
  $(x_2, y_2) := Maxmin(a_{m+1}, \ldots, a_n)$
  $x := \max(x_1, x_2)$
  $y := \min(y_1, y_2)$
return (x, y)

Finding the Maximum and Minimum

procedure *Maxmin*(a_1, …, a_n: real numbers with $n \geq 1$)
if $n = 1$ **then**
  $x := a_1$ $y := a_1$
else
  $m := \lfloor n/2 \rfloor$
  $(x_1, y_1) := Maxmin(a_1, \ldots, a_m)$
  $(x_2, y_2) := Maxmin(a_{m+1}, \ldots, a_n)$
  $x := \max(x_1, x_2)$
  $y := \min(y_1, y_2)$
return (x, y)

$f(n)$: the number of comparisons of *Maxmin*(a_1, …, a_n)

$f(n) = 2f(n/2) + 3$

if n is even

Merge Sort

procedure $mergesort(L = a_1, \ldots, a_n)$
if $n > 1$ **then**
$m := \lfloor n/2 \rfloor$
$L_1 := a_1, a_2, \ldots, a_m$
$L_2 := a_{m+1}, a_{m+2}, \ldots, a_n$
$L := merge(mergesort(L_1), \ mergesort(L_2))$
{L is now sorted into elements in nondecreasing order}

Merge Sort

procedure $mergesort(L = a_1, \ldots, a_n)$

if $n > 1$ **then**

$m := \lfloor n/2 \rfloor$

$L_1 := a_1, a_2, \ldots, a_m$

$L_2 := a_{m+1}, a_{m+2}, \ldots, a_n$

$L := merge(mergesort(L_1),\ mergesort(L_2))$

$\{L$ is now sorted into elements in nondecreasing order$\}$

$f(n)$: the number of comparisons of $mergesort(a_1, \ldots, a_n)$

$f(n) = 2f(n/2) + n$

if n is even

Fast Multiplication of Integers

A naive algorithm

$$A = a_{2n} \cdots a_1 \qquad B = b_{2n} \cdots b_1$$

$$A_1 = a_{2n} \cdots a_{n+1} \qquad B_1 = b_{2n} \cdots b_{n+1}$$

$$A_0 = a_n \cdots a_1 \qquad B_0 = b_n \cdots b_1$$

$$A = 2^n A_1 + A_0 \qquad B = 2^n B_1 + B_0$$

$$A \times B = 2^{2n} A_1 B_1 + 2^n (A_1 B_0 + A_0 B_1) + A_0 B_0$$

$f(n)$: Number of bit operations for n-bit inputs

$$f(2n) = 4f(n) + Cn$$

Fast Multiplication of Integers

Fast multiplication

$A = a_{2n} \cdots a_1$ $B = b_{2n} \cdots b_1$

$A_1 = a_{2n} \cdots a_{n+1}$ $B_1 = b_{2n} \cdots b_{n+1}$

$A_0 = a_n \cdots a_1$ $B_0 = b_n \cdots b_1$

$A = 2^n A_1 + A_0$ $B = 2^n B_1 + B_0$

$A \times B = (2^{2n} + 2^n)A_1B_1 + 2^n(A_1 - A_0)(B_0 - B_1) + (2^n + 1)A_0B_0$

$f(n)$: Number of bit operations for n-bit inputs

$f(2n) = 3f(n) + Cn$

Fast Multiplication of Matrices

$$A = [a_{ij}] \qquad B = [b_{ij}]$$

$$AB = [c_{ij}] \qquad c_{ij} = \sum_{k=1}^{n} a_{ik}\, b_{kj}$$

Naive algorithm: $\Theta(n^3)$

Number of multiplications/additions

Fast Multiplication of Matrices

$A = [a_{ij}]$ $B = [b_{ij}]$

A simple divide-and-conquer algorithm

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

Fast Multiplication of Matrices

$A = [a_{ij}]$ $B = [b_{ij}]$

A simple divide-and-conquer algorithm

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

$$\begin{aligned} C_{11} &= A_{11} \cdot B_{11} + A_{12} \cdot B_{21} , \\ C_{12} &= A_{11} \cdot B_{12} + A_{12} \cdot B_{22} , \\ C_{21} &= A_{21} \cdot B_{11} + A_{22} \cdot B_{21} , \\ C_{22} &= A_{21} \cdot B_{12} + A_{22} \cdot B_{22} . \end{aligned}$$

Fast Multiplication of Matrices

$A = [a_{ij}]$ $B = [b_{ij}]$

A simple divide-and-conquer algorithm

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

Complexity analysis

$f(n) = 8f(n/2) + n^2$

if n is even

Fast Multiplication of Matrices

$A = [a_{ij}]$ $B = [b_{ij}]$

Strassen's method

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$n/2$ $n/2$

$$\begin{aligned}
S_1 &= B_{12} - B_{22}\,, & S_6 &= B_{11} + B_{22}\,, \\
S_2 &= A_{11} + A_{12}\,, & S_7 &= A_{12} - A_{22}\,, \\
S_3 &= A_{21} + A_{22}\,, & S_8 &= B_{21} + B_{22}\,, \\
S_4 &= B_{21} - B_{11}\,, & S_9 &= A_{11} - A_{21}\,, \\
S_5 &= A_{11} + A_{22}\,, & S_{10} &= B_{11} + B_{12}\,.
\end{aligned}$$

Volker Strassen (1936-): German mathematician

Fast Multiplication of Matrices

$A = [a_{ij}]$ $B = [b_{ij}]$

Strassen's method

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$n/2$ $n/2$

$$\begin{aligned}
S_1 &= B_{12} - B_{22}, & S_6 &= B_{11} + B_{22}, \\
S_2 &= A_{11} + A_{12}, & S_7 &= A_{12} - A_{22}, \\
S_3 &= A_{21} + A_{22}, & S_8 &= B_{21} + B_{22}, \\
S_4 &= B_{21} - B_{11}, & S_9 &= A_{11} - A_{21}, \\
S_5 &= A_{11} + A_{22}, & S_{10} &= B_{11} + B_{12}.
\end{aligned}$$

$10 * n^2/4 = 5n^2/2$

Volker Strassen (1936-): German mathematician

Fast Multiplication of Matrices

$A = [a_{ij}]$ $B = [b_{ij}]$

Strassen's method

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$n/2$ $n/2$

7

$$\begin{aligned}
P_1 &= A_{11} \cdot S_1 = A_{11} \cdot B_{12} - A_{11} \cdot B_{22}\,, \\
P_2 &= S_2 \cdot B_{22} = A_{11} \cdot B_{22} + A_{12} \cdot B_{22}\,, \\
P_3 &= S_3 \cdot B_{11} = A_{21} \cdot B_{11} + A_{22} \cdot B_{11}\,, \\
P_4 &= A_{22} \cdot S_4 = A_{22} \cdot B_{21} - A_{22} \cdot B_{11}\,, \\
P_5 &= S_5 \cdot S_6 = A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22}\,, \\
P_6 &= S_7 \cdot S_8 = A_{12} \cdot B_{21} + A_{12} \cdot B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22}\,, \\
P_7 &= S_9 \cdot S_{10} = A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12}\,.
\end{aligned}$$

Fast Multiplication of Matrices

$A = [a_{ij}]$ $B = [b_{ij}]$

Strassen's method

$n/2$ $n/2$

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$7T(n/2)$ (7)

$$\begin{aligned}
P_1 &= A_{11} \cdot S_1 = A_{11} \cdot B_{12} - A_{11} \cdot B_{22}\,, \\
P_2 &= S_2 \cdot B_{22} = A_{11} \cdot B_{22} + A_{12} \cdot B_{22}\,, \\
P_3 &= S_3 \cdot B_{11} = A_{21} \cdot B_{11} + A_{22} \cdot B_{11}\,, \\
P_4 &= A_{22} \cdot S_4 = A_{22} \cdot B_{21} - A_{22} \cdot B_{11}\,, \\
P_5 &= S_5 \cdot S_6 = A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22}\,, \\
P_6 &= S_7 \cdot S_8 = A_{12} \cdot B_{21} + A_{12} \cdot B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22}\,, \\
P_7 &= S_9 \cdot S_{10} = A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12}\,.
\end{aligned}$$

Fast Multiplication of Matrices

$A = [a_{ij}]$ $B = [b_{ij}]$

Strassen's method

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$$C_{11} = P_5 + P_4 - P_2 + P_6$$

$$C_{12} = P_1 + P_2$$

$$C_{21} = P_3 + P_4$$

$$C_{22} = P_5 + P_1 - P_3 - P_7$$

Fast Multiplication of Matrices

$A = [a_{ij}]$ $B = [b_{ij}]$

Strassen's method

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

$n/2$ $n/2$

$$C_{11} = P_5 + P_4 - P_2 + P_6 \qquad 3n^2/4 +$$
$$C_{12} = P_1 + P_2 \qquad n^2/4 +$$
$$C_{21} = P_3 + P_4 \qquad n^2/4 +$$
$$C_{22} = P_5 + P_1 - P_3 - P_7 \qquad 3n^2/4 = 2n^2$$

Fast Multiplication of Matrices

$A = [a_{ij}]$ $B = [b_{ij}]$

Strassen's method

Complexity analysis

$$f(n) = 7f(n/2) + 9n^2/2$$

if n is even

Outline

- Solving linear nonhomogeneous recurrence relations
- Recurrence relations resulting from Divide-and-Conquer (DAC) algorithms
- **Solving DAC recurrence relations**
- Closest-Pair Problem

Solving DAC Recurrence Relations

$$f(n) = a\, f(n/b) + g(n)$$

Suppose $n = b^k$.

$$\begin{aligned}
f(n) &= af(n/b) + g(n) \\
&= a(af(n/b^2) + g(n/b)) + g(n) = a^2 f(n/b^2) + ag(n/b) + g(n) \\
&= a^3 f(n/b^3) + a^2 g(n/b^2) + ag(n/b) + g(n) \\
&= \dots \\
&= a^k f(n/b^k) + \sum_{j=0}^{k-1} a^j g(n/b^j) \\
&= a^k f(1) + \sum_{j=0}^{k-1} a^j g(n/b^j)
\end{aligned}$$

Solving DAC Recurrence Relations

$f(n) \le f(n')$ for every $n \le n'$ **Theorem**.

Let f be an increasing function satisfying $f(n) = a\, f(n/b) + c$ for every $n \in \mathbb{Z}_+$ divisible by b, where $a \in \mathbb{R}$ with $a \ge 1$, $b \in \mathbb{Z}$ with $b > 1$, and $c \in \mathbb{R}_+$. Then $f(n)$ is

$$\begin{cases} O(n^{\log_b a}) & \text{if } a > 1, \\ O(\log n) & \text{if } a = 1. \end{cases}$$

Moreover, if $n = b^k$ with $k \in \mathbb{Z}_+$ and $a > 1$, then

$$f(n) = C_1\, n^{\log_b a} + C_2,$$

where $C_1 = f(1) + c / (a\text{-}1)$, $C_2 = - c / (a\text{-}1)$

Solving DAC Recurrence Relations

Proof. Suppose $n = b^k$ with $k \in \mathbb{Z}_+$ and $a > 1$.

Then $f(n) = a^k f(1) + \sum_{j=0}^{k-1} a^j c = a^k f(1) + c\,\frac{a^k - 1}{a - 1}$

$= C_1\, a^k + C_2 = C_1\, a^{\log_b n} + C_2 = C_1\, n^{\log_n a \log_b n} + C_2$

$= C_1\, n^{\log_b a} + C_2$,

where $C_1 = f(1) + c / (a\text{-}1)$, $C_2 = - c / (a\text{-}1)$.

If $n = b^k$ and $a = 1$, then

$$f(n) = a^k f(1) + \sum_{j=0}^{k-1} a^j c = f(1) + ck = O(\log_b n) = O(\log n).$$

Solving DAC Recurrence Relations

Proof (*continued*).

Now let us consider $n \in \mathbb{Z}_+$.

Because $b > 1$, for each $n \geq b$, there exists $k \in \mathbb{Z}_+$ such that $b^k \leq n < b^{k+1}$.

Then $k \leq \log_b n < k+1$, thus $\log_b n - 1 < k \leq \log_b n$.

Since f is increasing, for each $n \geq b$, we have $f(n) \leq f(b^{k+1})$.

- If $a = 1$, then for each $n \geq b$, $f(n) \leq f(1) + c(k + 1)$. Thus

$$f(n) \leq f(1) + c(k + 1) \leq f(1) + c(\log_b n + 1).$$

Therefore, in this case, $f(n)$ is $O(\log n)$.

- If $a > 1$, then for each $n \geq b$, $f(n) \leq C_1 a^{k+1} + C_2 \leq C_1 a^{\log_b n + 1} + C_2$.

Then $f(n) \leq C_1 a\, n^{\log_b a} + C_2$.

Therefore, in this case, $f(n)$ is $O(n^{\log_b a})$.

Solving DAC Recurrence Relations

Examples

Binary search

Consider the recurrence relation $f(n) = f(n/2) + 3$, where n is even. Moreover, f is increasing.

Find the maximum and minimum in a sequence

Consider the recurrence relation $f(n) = 2f(n/2) + 3$, where n is even. Moreover, f is increasing.

Solving DAC Recurrence Relations

Master Theorem.

Let f be an increasing function satisfying $f(n) = a\, f(n/b) + cn^d$ for every $n \in \mathbb{Z}_+$ divisible by b, where $a \in \mathbb{R}$ with $a \geq 1$, $b \in \mathbb{Z}$ with $b > 1$, $c \in \mathbb{R}_+$, $d \in \mathbb{R}$ with $d \geq 0$. Then $f(n)$ is

$$\begin{cases} O(n^d) & \text{if } a < b^d, \\ O(n^d \log n) & \text{if } a = b^d, \\ O(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Solving DAC Recurrence Relations

Proof.

We first assume $n = b^k$. Then

$$f(n) = a^k f(1) + \sum_{j=0}^{k-1} a^j c(n/b^j)^d = a^k f(1) + cn^d \sum_{j=0}^{k-1} (a/b^d)^j.$$

- If $a < b^d$, then

$$f(n) = a^k f(1) + cn^d \frac{1 - (a/b^d)^k}{1 - a/b^d} < a^k f(1) + \frac{cn^d}{1 - a/b^d}.$$

Because $a^k \leq b^{dk} = (b^k)^d = n^d$, we deduce

$f(n) < n^d f(1) + \dfrac{cn^d}{1 - a/b^d}$. Thus $f(n)$ is $O(n^d)$.

Solving DAC Recurrence Relations

Proof (*continued*).

- If $a = b^d$, then

  $f(n) = a^k f(1) + cn^d k = b^{kd} f(1) + cn^d k = n^d f(1) + cn^d \log_b n$

  Thus $f(n)$ is $O(n^d \log n)$.

- If a > b^d, then

$$f(n) = a^k f(1) + cn^d \frac{1 - (a/b^d)^k}{1 - a/b^d} \leq a^k f(1) + c/(a/b^d - 1)\ n^d (a/b^d)^k$$

$$= a^{\log_b n} f(1) + c/(a/b^d - 1)\ n^d\ (a/b^d)^{\log_b n}$$

$$= n^{\log_b a} f(1) + c/(a/b^d - 1)\ n^d\ n^{\log_b (a/b^d)}$$

$$= n^{\log_b a} f(1) + c/(a/b^d - 1) n^{\log_b a}.$$

Thus $f(n)$ is $O(n^{\log_b a})$.

Solving DAC Recurrence Relations

Proof (*continued*).

Next, we assume $n \in \mathbb{Z}_+$.

Let $n \geq b$. Then there is $k \in \mathbb{Z}_+$ such that $b^k \leq n < b^{k+1}$.

Because f is increasing, $f(n) \leq f(b^{k+1})$.

- If $a < b^d$, then

$$f(n) \leq a^{k+1} f(1) + cn^d \frac{1 - (a/b^d)^{k+1}}{1 - a/b^d} < a^{k+1} f(1) + \frac{cn^d}{1 - a/b^d}.$$

From $b^k \leq n < b^{k+1}$, we know that $\log_b n - 1 < k \leq \log_b n$.

Thus $a^{k+1} f(1) + \frac{cn^d}{1 - a/b^d} < b^{d(k+1)} f(1) + \frac{cn^d}{1 - a/b^d} \leq$

$$b^{d(\log_b n + 1)} f(1) + \frac{cn^d}{1 - a/b^d} \leq b^d n^d f(1) + \frac{cn^d}{1 - a/b^d}.$$

In this case, we deduce that $f(n)$ is $O(n^d)$.

Similarly for the case $a = b^d$ and $a > b^d$.

Solving DAC Recurrence Relations

Examples

Merge Sort

Consider the recurrence relation $f(n) = 2f(n/2) + n$, where n is even. Moreover, f is increasing.

Solving DAC Recurrence Relations

Examples

Multiplication of integers

Consider the recurrence relation $f(n) = 4f(n/2) + Cn$, where n is even. Moreover, f is increasing.

Fast multiplication of integers

Consider the recurrence relation $f(n) = 3f(n/2) + Cn$, where n is even. Moreover, f is increasing.

Solving DAC Recurrence Relations

Examples

Fast multiplication of matrices

Consider the recurrence relation $f(n) = 7f(n/2) + 9n^2/4$, where n is even. Moreover, f is increasing.

Outline

- Solving linear nonhomogeneous recurrence relations
- Recurrence relations resulting from Divide-and-Conquer (DAC) algorithms
- Solving DAC recurrence relations
- **Closest-Pair Problem**

Closest-Pair Problem

Given *n* points $(x_1, y_1), \cdots, (x_n, y_n)$ in the plane,

find the closed pair of points, namely,

find *i,j*: $1 \leq i < j \leq n$ such that

the Euclidean distance between (x_i, y_i) and (y_i, y_j),

i.e. $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$, is minimal.

The naive algorithm

Compute the distances for all pairs and find the minimum.

Complexity: $C_n^2 = \Theta(n^2)$ distance computations.

Closest-Pair Problem

The fast algorithm (Assume $n=2^k$ for simplicity)

Phase I.

Mergesort the points according to the *x* coordinate and obtain the sequence $(x_{i_1}, y_{i_1}), \cdots, (x_{i_n}, y_{i_n})$.

Mergesort the points according to the *y* coordinate and obtain the sequence $(x_{j_1}, y_{j_1}), \cdots, (x_{j_n}, y_{j_n})$.

(1.5, 2) (1, 3) (7, 3) (5, 1)

Mergesort according to *x*-coordinate: (1, 3) (1.5, 2) (5, 1) (7, 3)

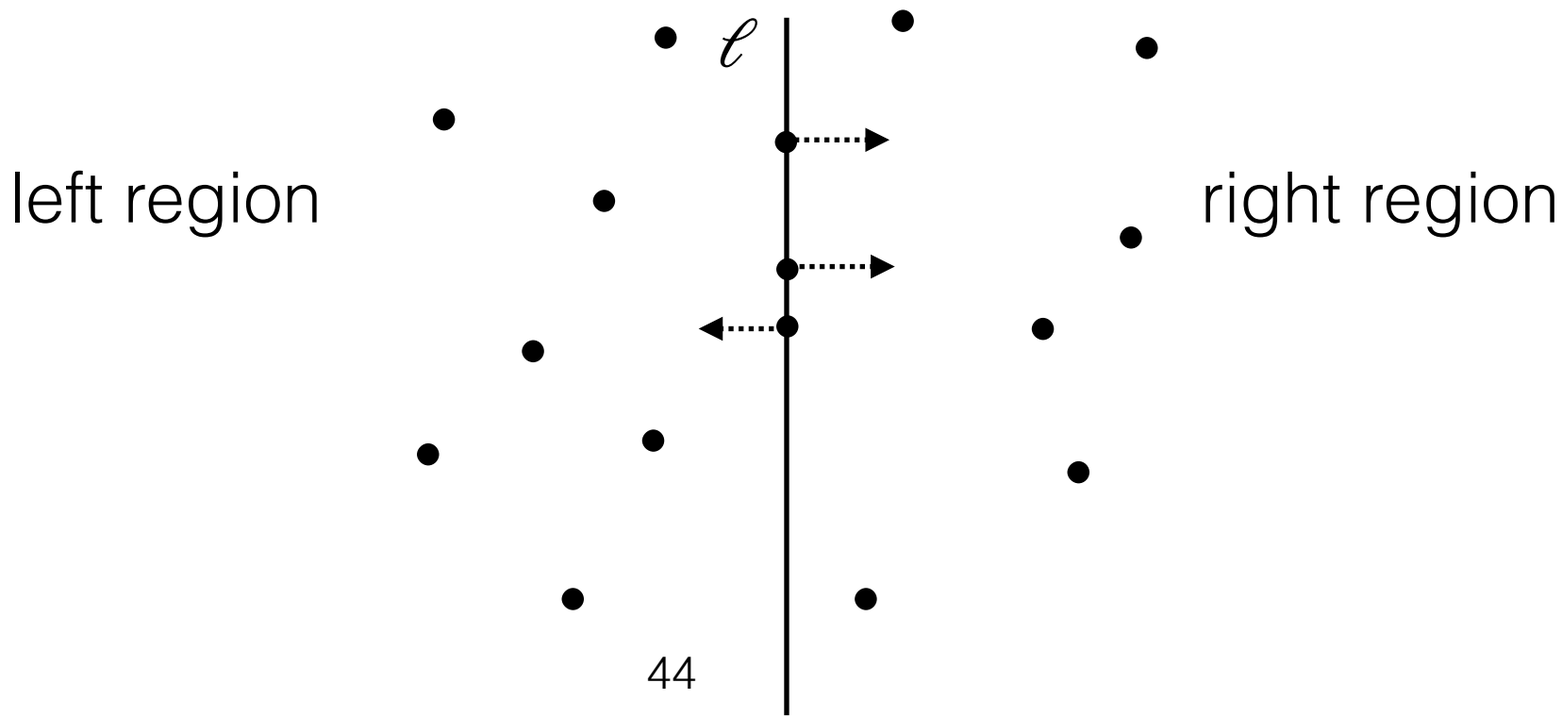
Mergesort according to *y*-coordinate: (5, 1) (1.5, 2) (1, 3) (7, 3)

Closest-Pair Problem

The fast algorithm (Assume $n=2^k$ for simplicity)

Phase II. Use the sequence $(x_{i_1}, y_{i_1}), \cdots, (x_{i_n}, y_{i_n})$ to find a vertical line ℓ that divides the plane into two regions with equal number of points.
More precisely, find a vertical line ℓ such that

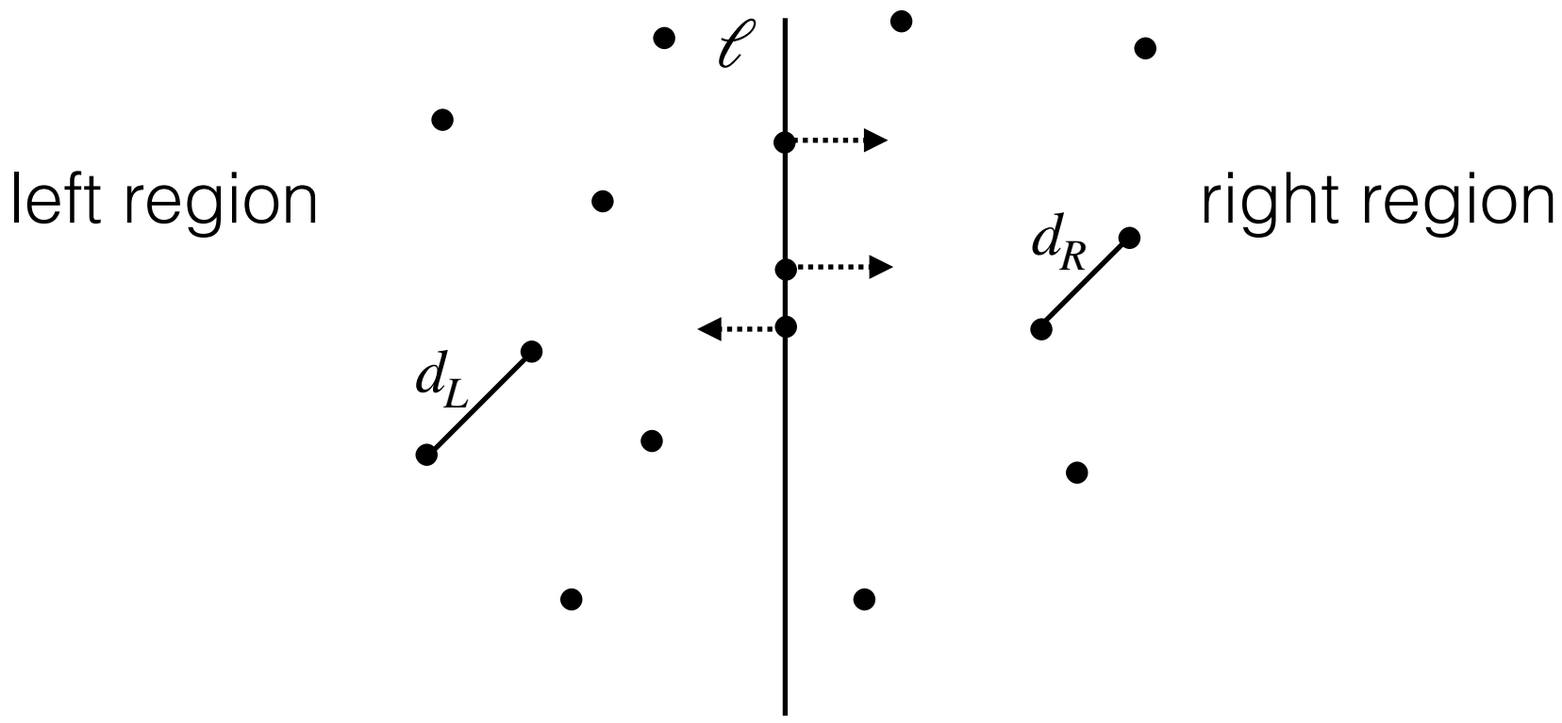
1. there is at least one point in ℓ,
2. the number of points to the left of ℓ (including those in ℓ) is $\geq n/2$,
3. the number of points to the right of ℓ (including those in ℓ) is $\geq n/2$.



Closest-Pair Problem

The fast algorithm (Assume $n=2^k$ for simplicity)

Phase III. Find (recursively) the two closest pairs of the left and right region respectively.
Let d_L and d_R denote the minimum distances in the left and right region.
Moreover, let $d = \min(d_L, d_R)$.

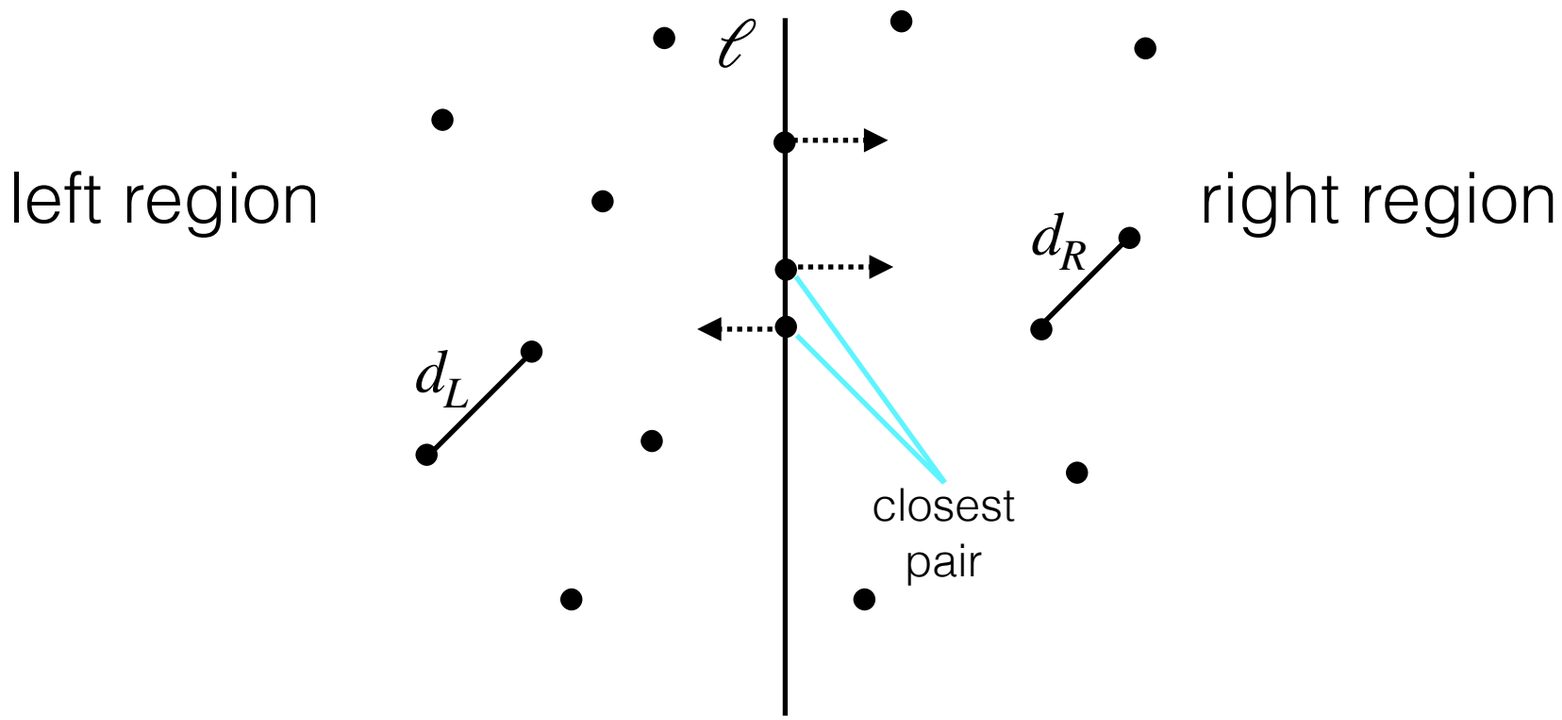


Closest-Pair Problem

The fast algorithm (Assume $n=2^k$ for simplicity)

The closest pair may be

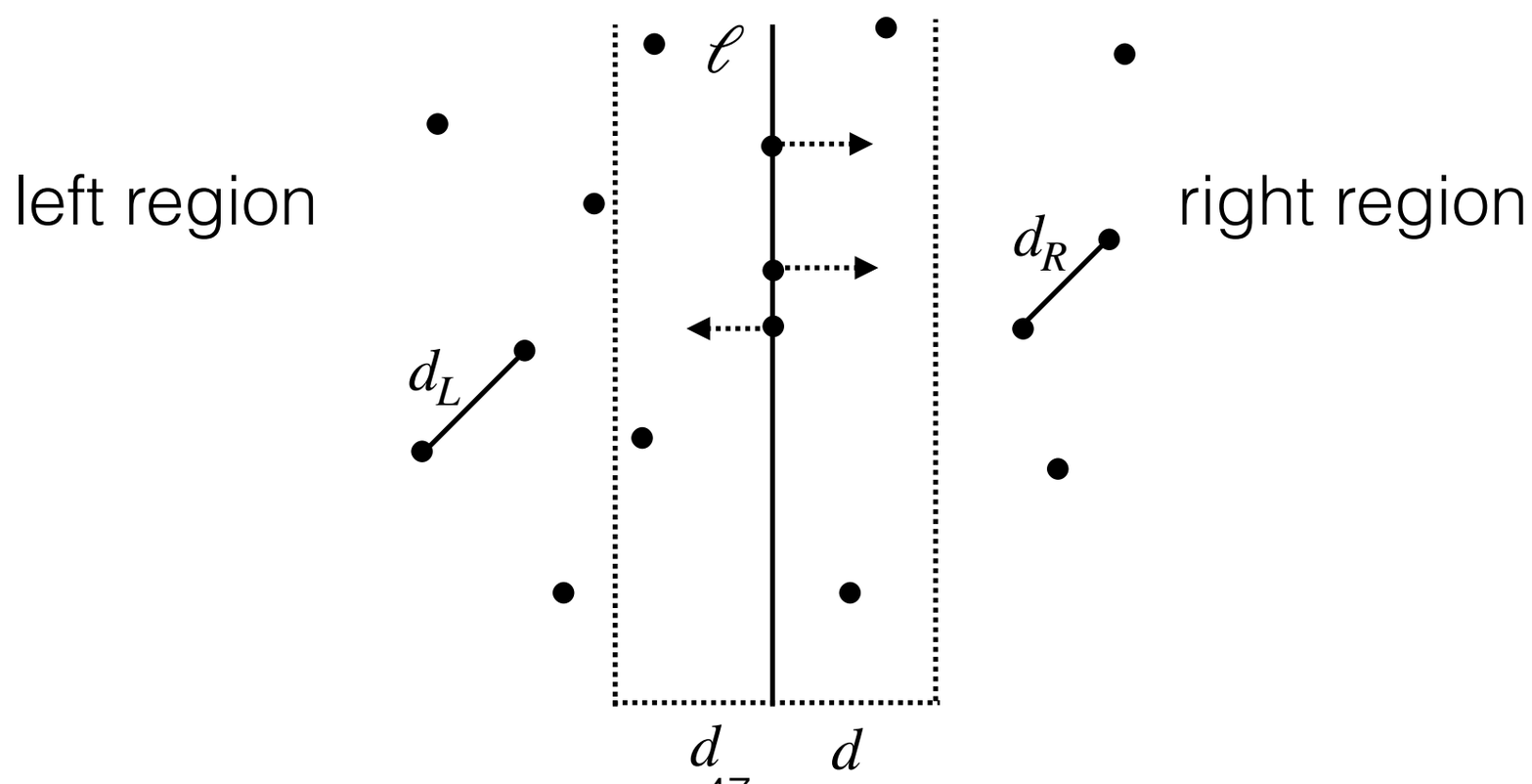
- either the closest pair in the left region,
- the closest pair in the right region,
- or a pair crossing the two regions.



Closest-Pair Problem

The fast algorithm (Assume $n=2^k$ for simplicity)

For computing the closest pair crossing the two regions, it is sufficient to consider the points in the vertical strip centered at ℓ of width $2d$.



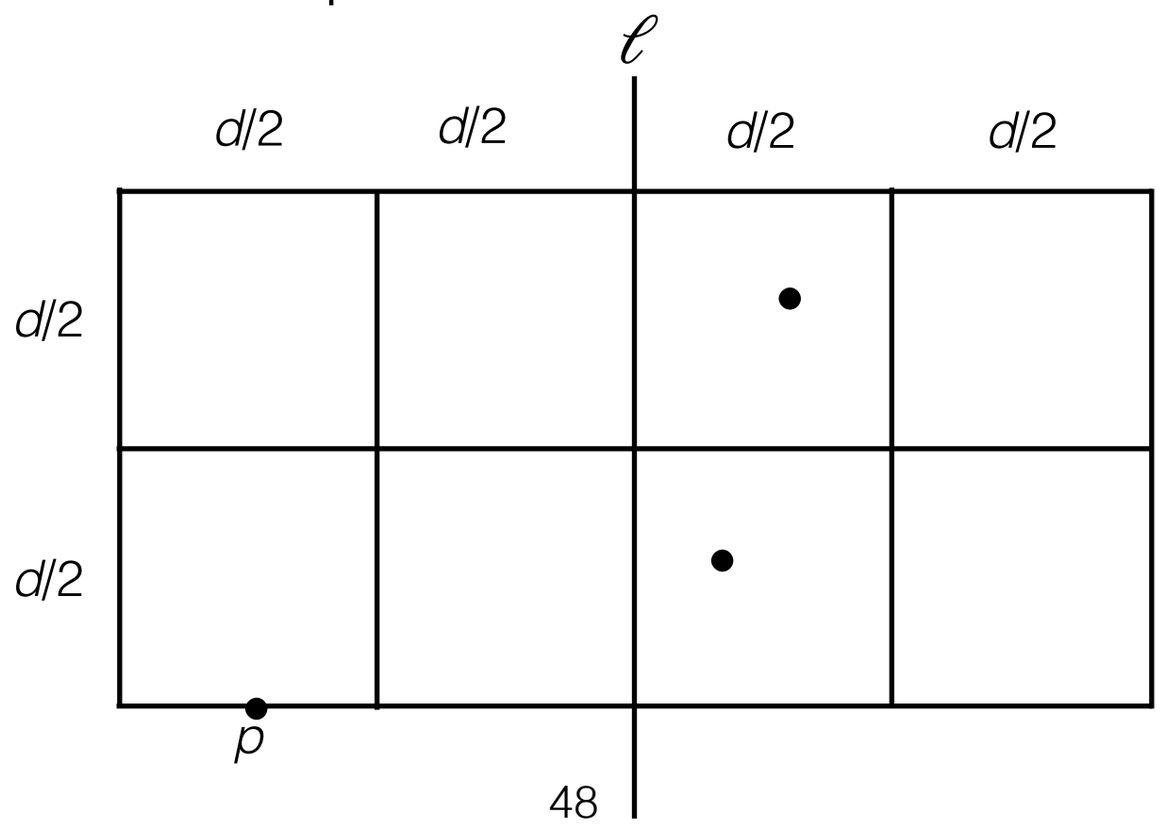
Closest-Pair Problem

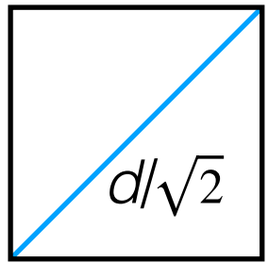
The fast algorithm (Assume $n=2^k$ for simplicity)

Phase IV. Use the sequence $(x_{j_1}, y_{j_1}), \cdots, (x_{j_n}, y_{j_n})$ to compute for each point *p* in the vertical strip (points of smaller *y*-coordinates are considered sooner), the distances between *p* and the points in the rectangle of width 2*d* and height *d* where *p* belongs to the bottom border.

Claim. Sufficient to compute at most four distances for *p*, in fact.

In this case, two distances are enough





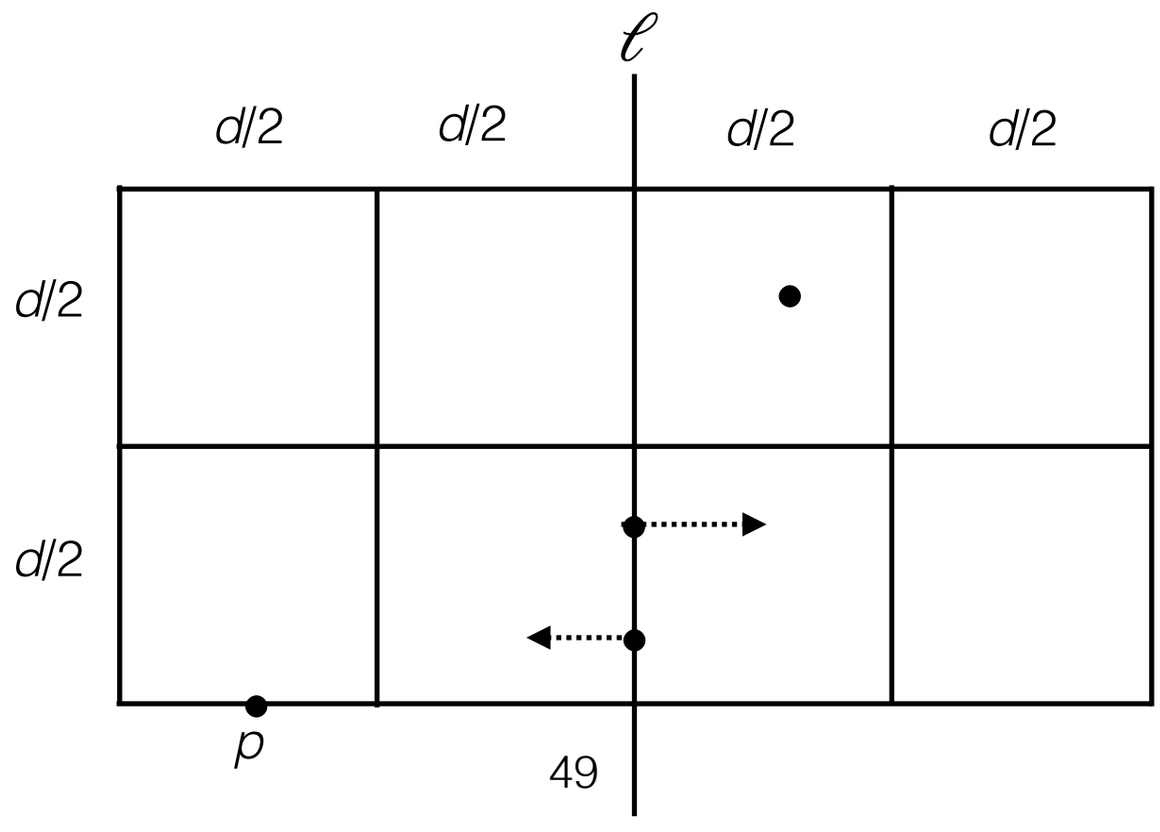
Closest-Pair Problem

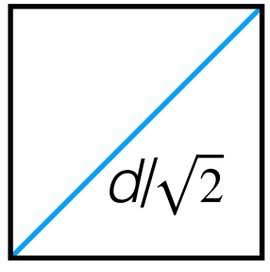
The fast algorithm (Assume $n=2^k$ for simplicity)

Phase IV. Use the sequence $(x_{j_1}, y_{j_1}), \cdots, (x_{j_n}, y_{j_n})$ to compute for each point *p* in the vertical strip (points of smaller *y*-coordinates are considered sooner), the distances between *p* and the points in the rectangle of width $2d$ and height d where *p* belongs to the bottom border.

Claim. Sufficient to compute at most four distances for *p*, in fact.

In this case, two distances are enough





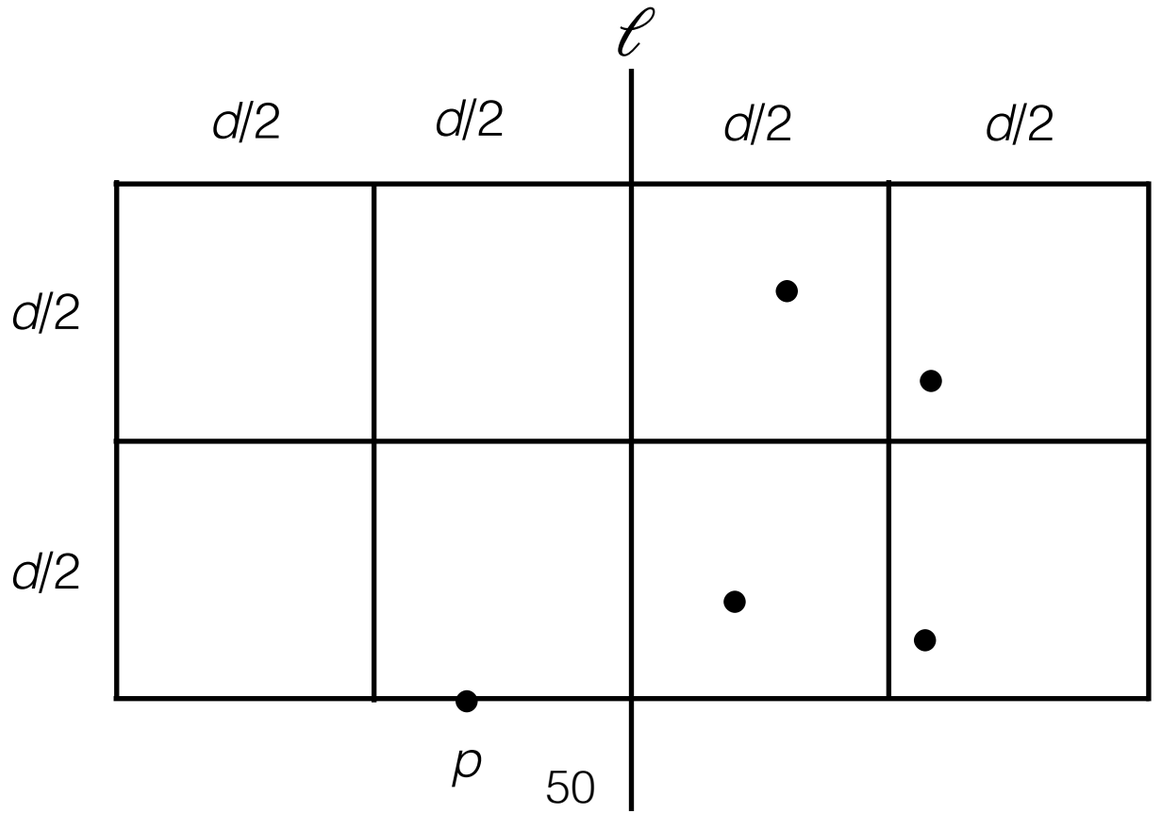
Closest-Pair Problem

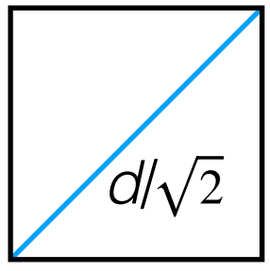
The fast algorithm (Assume $n=2^k$ for simplicity)

Phase IV. Use the sequence $(x_{j_1}, y_{j_1}), \cdots, (x_{j_n}, y_{j_n})$ to compute for each point *p* in the vertical strip (points of smaller *y*-coordinates are considered sooner), the distances between *p* and the points in the rectangle of width $2d$ and height d where *p* belongs to the bottom border.

Claim. Sufficient to compute at most four distances for *p*, in fact.

In this case, four distances are enough



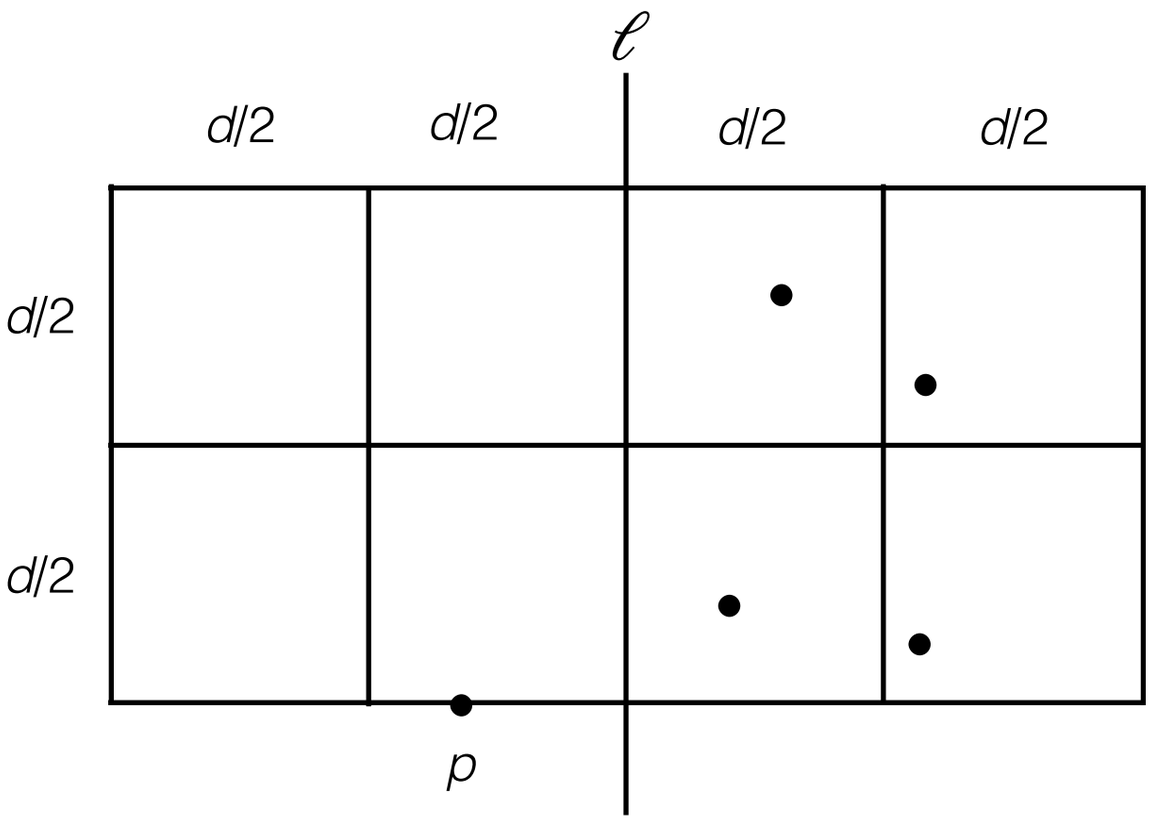


Closest-Pair Problem

The fast algorithm (Assume $n=2^k$ for simplicity)

Phase IV. Use the sequence $(x_{j_1}, y_{j_1}), \cdots, (x_{j_n}, y_{j_n})$ to compute for each point *p* in the vertical strip (points of smaller *y*-coordinates are considered sooner), the distances between *p* and the points in the rectangle of width 2*d* and height *d* where *p* belongs to the bottom border.

Claim. Sufficient to compute at most four distances for *p*, in fact.



f(*n*):
number of distance computations
for **Phase II-IV**.

$$f(n) = 2f(n/2) + 4n$$

Closest-Pair Problem

Consider the recurrence relation

$f(n) = 2f(n/2) + 4n$, where n is even.

Moreover, f is increasing.

Since $a = 2$, $b = 2$, d=1, we have $a = b^d$.

From the master theorem,

$f(n)$ is $O(n^d \log n) = O(n \log n)$.

Theorem.

Let $a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k} + f(n)$ with $c_1, \ldots, c_k \in \mathbb{R}$ be a linear nonhomogeneous recurrence relation and $\{h_n^{(p)}\}$ be one of its solutions. Then every solution of $a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k} + f(n)$ is of the form $\{h_n^{(p)} + h_n^{(q)}\}$, where $\{h_n^{(q)}\}$ is a solution of its associated homogeneous recurrence relation.

Theorem.

Let $a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k} + f(n)$ with $c_1, \ldots, c_k \in \mathbb{R}$ be a linear nonhomogeneous recurrence relation. Moreover, suppose $f(n) = (b_t n^t + b_{t-1} n^{t-1} + \cdots + b_1 n + b_0) s^n$ with $b_t, \ldots, b_0, s \in \mathbb{R}$.

If s is not the root of the characteristic equation of $a_n = c_1\, a_{n-1} + c_2\, a_{n-2} + \ldots + c_k\, a_{n-k}$, then there is a solution of the nonhomogeneous recurrence relation of the form $\{(p_t n^t + p_{t-1} n^{t-1} + \cdots + p_1 n + p_0) s^n\}$, otherwise, let m be the multiplicity of s, then there is a solution of the form $\{n^m (p_t n^t + p_{t-1} n^{t-1} + \cdots + p_1 n + p_0) s^n\}$, where $p_t, \ldots, p_0 \in \mathbb{R}$.

Recap

- Solving linear nonhomogeneous recurrence relations with constant coefficients
- Recurrence relations resulting from Divide-and-Conquer (DAC) algorithms
- Solving the DAC recurrence equations
- Closed-pair problem

$f(n) \leq f(n')$ for every $n \leq n'$ **Theorem**.

Let f be an increasing function satisfying $f(n) = a\, f(n/b) + c$ for every $n \in \mathbb{Z}^+$ divisible by b, where $a \in \mathbb{R}$ with $a \geq 1$, $b \in \mathbb{Z}$ with $b > 1$, and $c \in \mathbb{R}^+$. Then $f(n)$ is

$$\begin{cases} O(n^{\log_b a}) & \text{if } a > 1, \\ O(\log n) & \text{if } a = 1. \end{cases}$$

Moreover, if $n = b^k$ with $k \in \mathbb{Z}^+$ and $a > 1$, then

$$f(n) = C_1\, n^{\log_b a} + C_2,$$

where $C_1 = f(1) + c / (a\text{-}1)$, $C_2 = - c / (a\text{-}1)$

Master Theorem.

Let f be a increasing function satisfying $f(n) = a\, f(n/b) + cn^d$ for every $n \in \mathbb{Z}^+$ divisible by b, where $a \in \mathbb{R}$ with $a \geq 1$, $b \in \mathbb{Z}$ with $b > 1$, $c \in \mathbb{R}^+$, $d \in \mathbb{R}$ with $d \geq 0$. Then $f(n)$ is

$$\begin{cases} O(n^d) & \text{if } a < b^d, \\ O(n^d \log n) & \text{if } a = b^d, \\ O(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Homework

1. Section 8.2.
   - Page 552, Exercise 30.
   - Use the two theorems for nonhomogeneous linear recurrence relations to solve the recurrence relation $a_n = a_{n-1} + n^3$, $a_0 = 0$.
2. Section 8.3.
   - Page 562, Exercise 22.
   - Page 563, Exercise 36, 37.

作业和答疑

1. 交作业时间：奇数周周一上课前（两周交一次）
2. 交作业方式（二选一）：
   - 通过https://mooc.ucas.edu.cn/portal课程网站
   - 上课现场提交
3. 答疑：助教崔乐天、刘易钺

Next Lecture

Generating Functions, Generalized Inclusion-Exclusion Principle